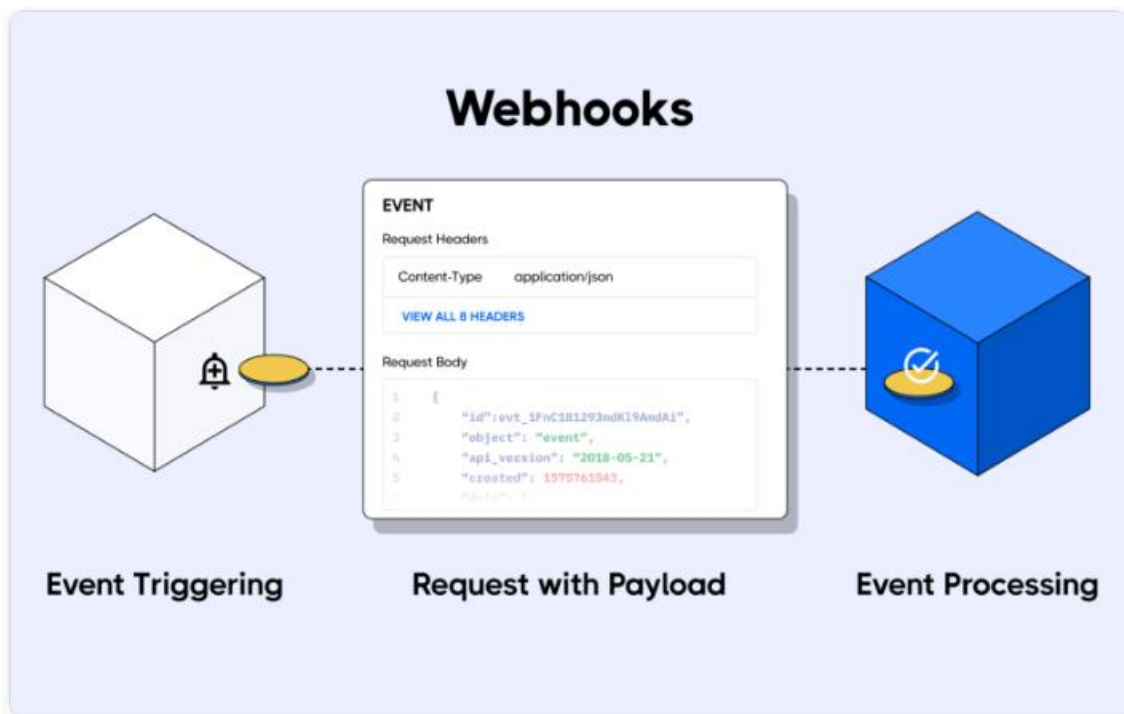




Jenkins

What Are Webhooks?

- A webhook allows one system to notify another in real time.
- It enables automatic communication when specific events happen.
- Webhooks help trigger actions without manual involvement.
- They are widely used in development, automation, and service integrations..



This is the way webhooks typically work:

Event Occurs:

- A specific action happens in the source system that triggers a notification.
- Examples include new commits, new issues, or user actions on a website.

Webhook Configuration:

- A webhook URL is set in the receiver system to accept event data.
- This URL acts as a callback location to receive notifications.

HTTP POST Request:

- When the event occurs, the source sends data to the webhook URL.
- The payload usually contains event type, details, and metadata in JSON/XML.

Receiver Processing:

- The receiver reads the incoming data and performs required actions.
- This may involve updating data, triggering workflows, or sending alerts.



Common Real-World Use Cases :-

Service / Platform	Purpose
GitHub	Trigger CI/CD pipelines on code push
PayPal / Stripe	Payment transaction updates
WhatsApp API	Message delivered & read notifications
Slack	Send messages to channels automatically
Shopify	Sync orders, stock, and shipping
Jenkins	Build automation on events

Advantages of Webhooks :-

1. Real-time communication
2. Lightweight mechanism
3. Low server load (no polling)
4. Great for automation workflows

Limitations of Webhooks :-

1. Not suitable for big/continuous data streams
2. Requires public & secure endpoints
3. Debugging failures can be complex
4. Delivery can fail if the receiver is down for too long

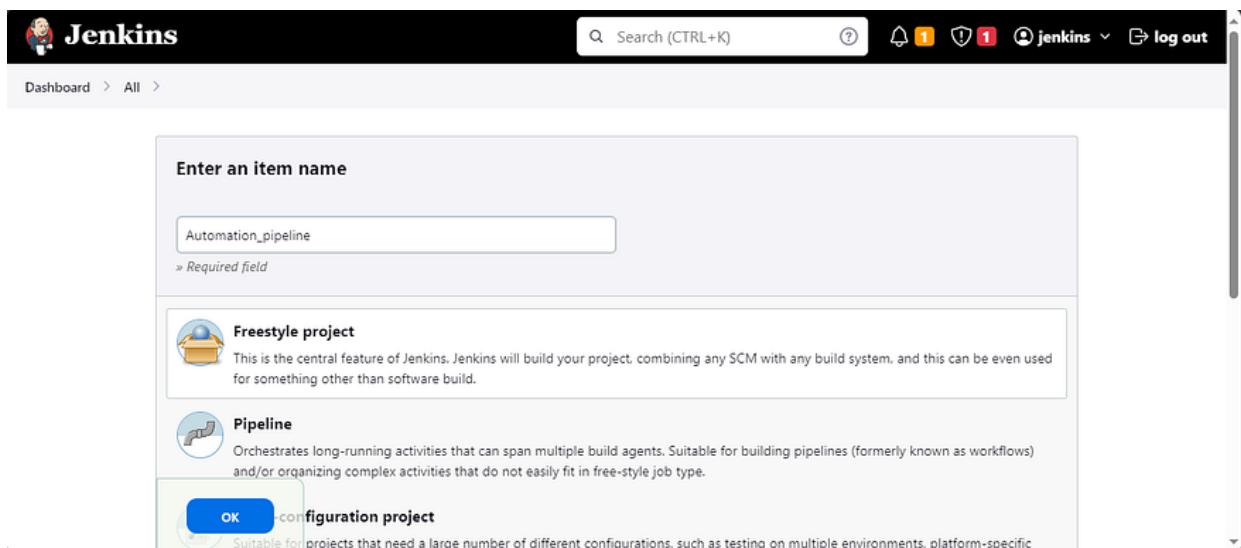
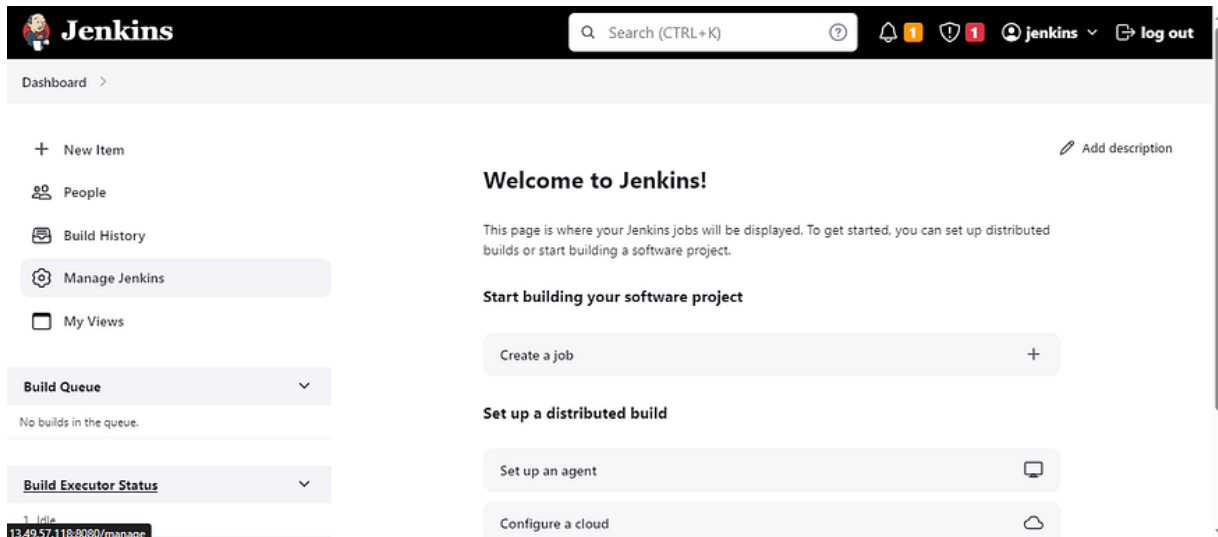
Webhooks vs. APIs :-

Aspect	Webhooks (Push)	APIs (Polling)
Data Delivery	Automatic, event-driven	Client requests data on demand
Real-Time	Instant updates	Delayed, based on polling interval
Control	Server decides timing and content	Client controls timing and volume
Data Efficiency	Low overhead, no unnecessary requests	High overhead due to frequent polling
Reliability	Risk of data loss if endpoint is down	Reliable, client pulls when ready
Failure Handling	Requires retry/queue mechanism	Not required
Best Use Case	Event notifications, payments, CI/CD alerts	Dashboards, analytics, bulk data retrieval
Complexity	Simple to implement for events	More flexible but may require more setup

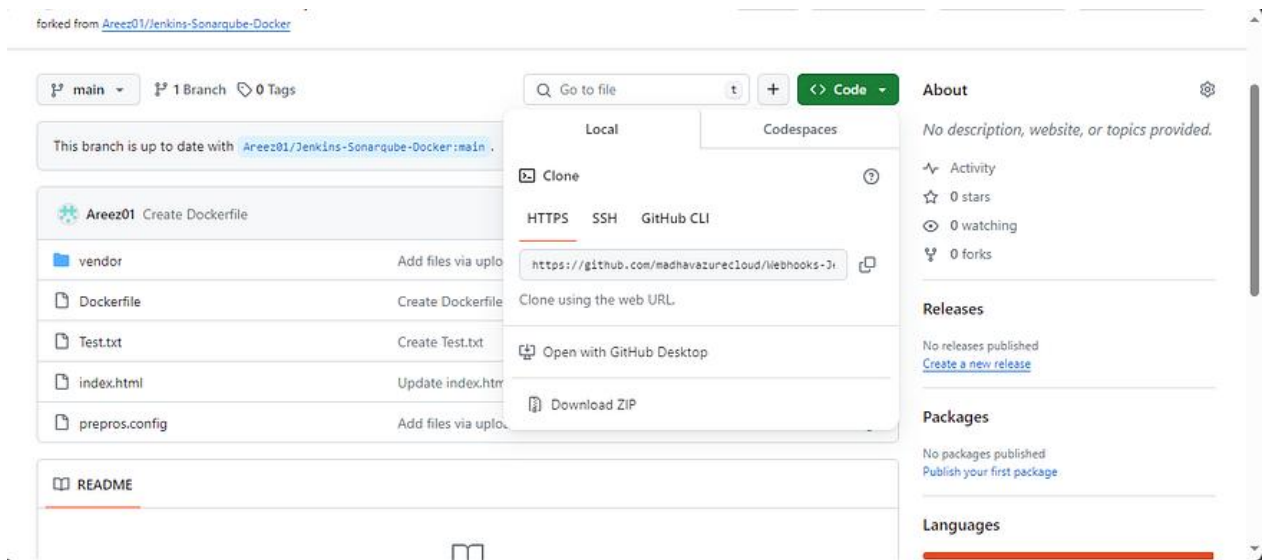
Steps to create jenkins Webhooks :-

TASK 1:

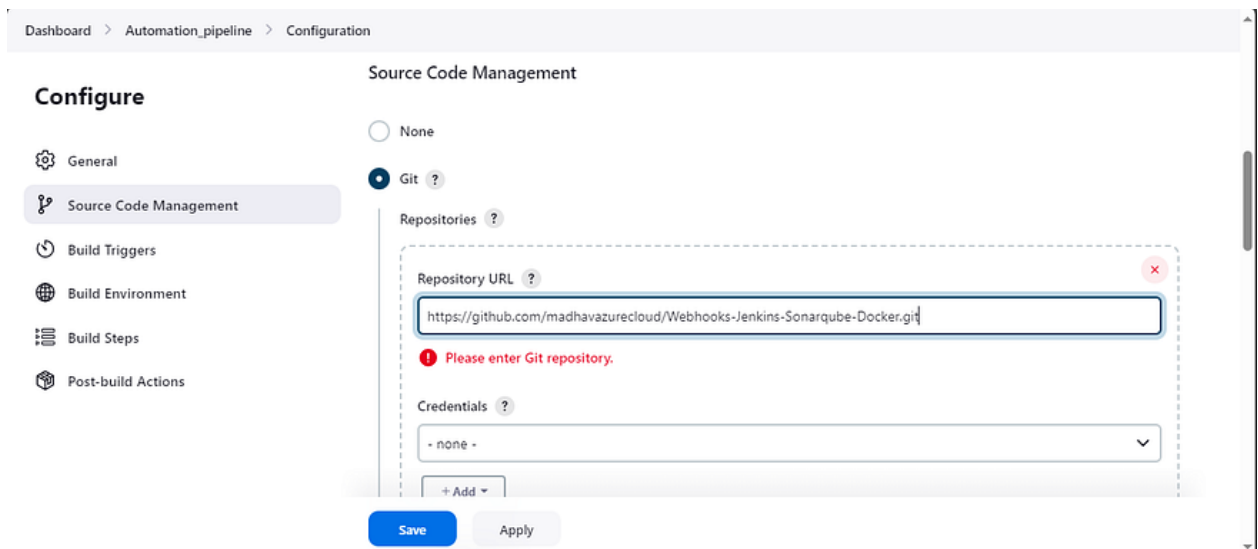
Step 1 : Open jenkins, Click +New item and create a Freestyle project.



Step 3: Copy the URL from GitHub and paste it in Jenkins



Step 4: Select Git and paste the URL



Step 5 : Change the Branch Specifier from master to main

Dashboard > Automation_pipeline > Configuration

Configure

- General
- Source Code Management
- Build Triggers
- Build Environment
- Build Steps
- Post-build Actions

Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

*/main

Add Branch

Repository browser ?

(Auto)

Additional Behaviours

Add

Save Apply

Step 6 : Select GitHub hook trigger

Dashboard > Automation_pipeline > Configuration

Configure

- General
- Source Code Management
- Build Triggers
- Build Environment
- Build Steps
- Post-build Actions

Build Triggers

- ☐ Trigger builds remotely (e.g., from scripts) ?
- ☐ Build after other projects are built ?
- ☐ Build periodically ?
- ☒ GitHub hook trigger for GITScm polling ?
- ☐ Poll SCM ?

Build Environment

- ☐ Delete workspace before build starts
- ☐ Use secret text(s) or file(s) ?
- ☐ Add timestamps to the Console Output

Save Apply

TASK 2:

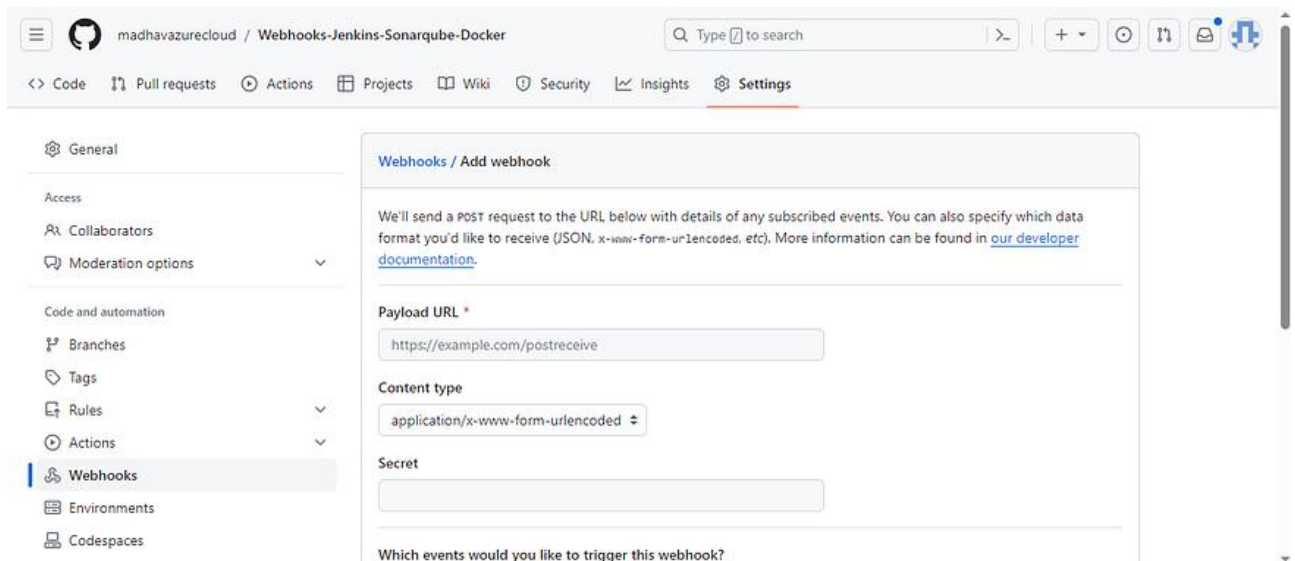
Step 7 : Click settings

The screenshot shows the GitHub repository page for 'Webhooks-Jenkins-Sonarqube-Docker' by user 'madhavazurecloud'. The repository is a fork of 'Areez01/Jenkins-Sonarqube-Docker'. The 'Settings' tab is selected in the top navigation bar. Below the repository name, there are buttons for 'Pin', 'Watch' (0), 'Fork' (0), and 'Star' (0). The main content area shows the repository's file structure: 'vendor', 'Dockerfile', 'Test.txt', and 'index.html'. The 'About' section on the right indicates no description, website, or topics are provided. The 'Releases' section shows no releases published. A URL bar at the bottom of the screenshot shows 'https://github.com/madhavazurecloud/Webhooks-Jenkins-Sonarqube-Docker/settings'.

Step 8 : Click Webhooks

The screenshot shows the 'Webhooks' settings page for the repository 'Webhooks-Jenkins-Sonarqube-Dock'. The left sidebar contains navigation options: 'Access', 'Collaborators', 'Moderation options', 'Code and automation', 'Branches', 'Tags', 'Rules', 'Actions', 'Webhooks' (highlighted with a red box), 'Environments', 'Codespaces', 'Pages', 'Security', 'Code security and analysis', 'Deploy keys', and 'Secrets and variables'. The main content area is titled 'Repository name' and shows the repository name 'Webhooks-Jenkins-Sonarqube-Dock' with a 'Rename' button. Below this, there are checkboxes for 'Template repository' and 'Require contributors to sign off on web-based commits'. The 'Default branch' section shows the default branch is 'main'. The 'Social preview' section has a heading and a description: 'Upload an image to customize your repository's social media preview. Images should be at least 640x320px (1280x640px for best display).'

Step 9 : Then click “add webhook”

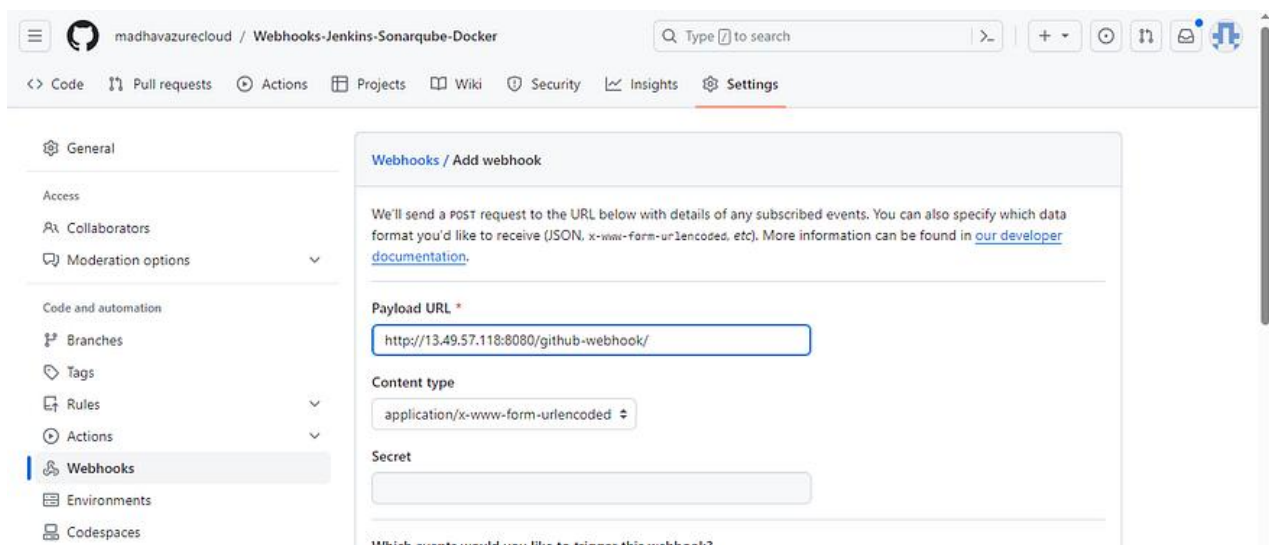


The screenshot shows the GitHub repository settings for 'madhavazurecloud / Webhooks-Jenkins-Sonarqube-Docker'. The 'Settings' tab is selected, and the 'Webhooks' section is highlighted in the left sidebar. The 'Add webhook' form is displayed with the following fields:

- Payload URL ***: `https://example.com/postreceive`
- Content type**: `application/x-www-form-urlencoded`
- Secret**: (empty field)

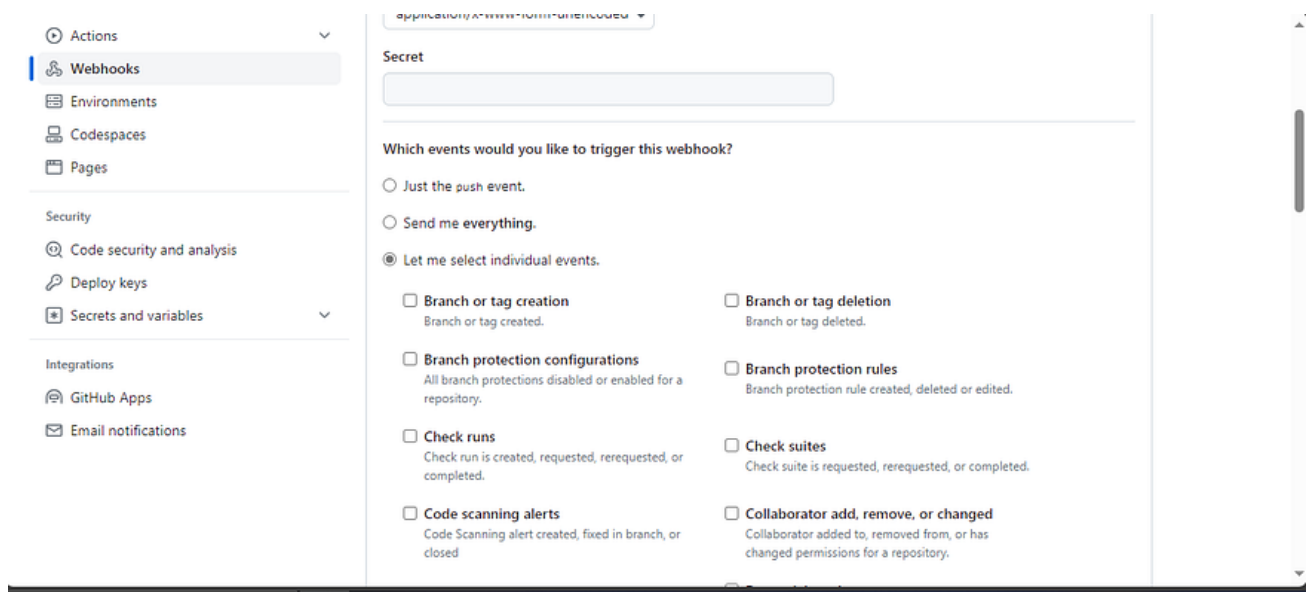
Below the form, there is a section titled 'Which events would you like to trigger this webhook?'.

Step 9 : Copy the jenkins URL with port number and paste it in the Payload URL



The screenshot shows the same GitHub repository settings page, but the 'Payload URL' field now contains the Jenkins URL: `http://13.49.57.118:8080/github-webhook/`. The other fields remain the same.

Step 10 : Select “Let me select individual events”



The screenshot shows the GitHub Webhook configuration interface. On the left is a sidebar with navigation links: Actions, Webhooks (selected), Environments, Codespaces, Pages, Security, Code security and analysis, Deploy keys, Secrets and variables, Integrations, GitHub Apps, and Email notifications. The main content area is titled 'Secret' and has a text input field. Below this, the question 'Which events would you like to trigger this webhook?' is followed by three radio button options: 'Just the push event.', 'Send me everything.', and 'Let me select individual events.' (which is selected). Under the selected option, there is a grid of checkboxes for various events: Branch or tag creation, Branch or tag deletion, Branch protection configurations, Branch protection rules, Check runs, Check suites, Code scanning alerts, and Collaborator add, remove, or changed. Each checkbox is currently unchecked.

application/vnd.github+json

Secret

Which events would you like to trigger this webhook?

☐ Just the push event.

☐ Send me everything.

☒ Let me select individual events.

☐ Branch or tag creation
Branch or tag created.

☐ Branch or tag deletion
Branch or tag deleted.

☐ Branch protection configurations
All branch protections disabled or enabled for a repository.

☐ Branch protection rules
Branch protection rule created, deleted or edited.

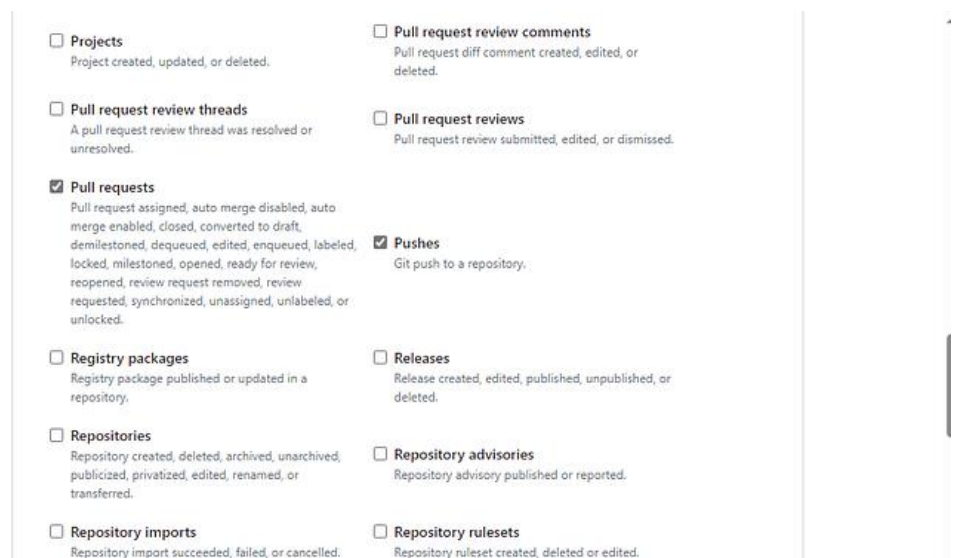
☐ Check runs
Check run is created, requested, rerequested, or completed.

☐ Check suites
Check suite is requested, rerequested, or completed.

☐ Code scanning alerts
Code Scanning alert created, fixed in branch, or closed

☐ Collaborator add, remove, or changed
Collaborator added to, removed from, or has changed permissions for a repository.

Step 10 : Select “Pull requests”



The screenshot shows the same GitHub Webhook configuration interface as before, but with the 'Pull requests' checkbox under the 'Let me select individual events' section now checked. All other checkboxes remain unchecked.

☐ Projects
Project created, updated, or deleted.

☐ Pull request review threads
A pull request review thread was resolved or unresolved.

☒ Pull requests
Pull request assigned, auto merge disabled, auto merge enabled, closed, converted to draft, demilestoned, dequeued, edited, enqueued, labeled, locked, milestoned, opened, ready for review, reopened, review request removed, review requested, synchronized, unassigned, unlabeled, or unlocked.

☐ Registry packages
Registry package published or updated in a repository.

☐ Repositories
Repository created, deleted, archived, unarchived, publicized, privatized, edited, renamed, or transferred.

☐ Repository imports
Repository import succeeded, failed, or cancelled.

☐ Pull request review comments
Pull request diff comment created, edited, or deleted.

☐ Pull request reviews
Pull request review submitted, edited, or dismissed.

☒ Pushes
Git push to a repository.

☐ Releases
Release created, edited, published, unpublished, or deleted.

☐ Repository advisories
Repository advisory published or reported.

☐ Repository rulesets
Repository ruleset created, deleted or edited.

Step 11 : Then Click “Add webhook”

or validated

☐ Stars

A star is created or deleted from a repository.

☐ Team adds

Team added or modified on a repository.

☐ Watches

User stars a repository.

☐ Workflow jobs

Workflow job queued, waiting, in progress, or completed on a repository.

☐ Statuses

Commit status updated from the API.

☐ Visibility changes

Repository changes from private to public.

☐ Wiki

Wiki page updated.

☐ Workflow runs

Workflow run requested or completed on a repository.

☒ Active

We will deliver event details when this hook is triggered.

Add webhook